

MAGNED

ESTÚDIO ACADÊMICO NEURAL

Roteiro Técnico de Apresentação

Como cada função foi construída — Detalhamento por integrante

Grupo 01 — Engenharia de Software · UniEVANGÉLICA
Augusto · Gabriel · Eduardo · Nicolas · Davi · Marcel

Ciclo 3 — Junho 2026

AU

Augusto

Abertura · Arquitetura · Reconhecimento Facial

 Ação: Abre index.html → Faz Login Facial ao vivo

"Nosso projeto é o **MAGNED**. A arquitetura é **Client-Centric**: tudo roda no navegador. São apenas **6 arquivos** no total. Não existe backend, não existe servidor Node ou Python rodando. Isso torna o projeto **custo zero** e com **latência zero**."

Como funciona o Reconhecimento Facial (FaceID)

- Usamos a biblioteca `face-api.js`, que é construída em cima do **TensorFlow.js**.
- São carregados 3 modelos de IA da CDN: `tinyFaceDetector` (detecta onde está o rosto), `faceLandmark68Net` (mapeia 68 pontos do rosto: olhos, nariz, boca, queixo), e `faceRecognitionNet` (gera um vetor numérico de 128 dimensões — um "DNA facial").
- **Cadastro:** A função `startFaceRegister()` abre a câmera via `navigator.mediaDevices.getUserMedia()`. Um loop (`setInterval`) roda a cada 500ms, capturando frames do vídeo e extraindo o vetor facial. Coletamos **2+ amostras** que são salvas como `Float32Array` no objeto de estado `S.faceDesc`.
- **Login:** A função `startFaceLogin()` faz o mesmo loop, mas ao invés de salvar, ela **compara** o vetor atual com os vetores salvos usando `faceapi.euclideanDistance()`. Se a distância for **menor que 0.5**, a identidade é confirmada e o aluno é logado automaticamente.

Onde ficam os dados? (Banco de Dados)

- **Localmente:** Tudo é salvo no `localStorage` do navegador com a chave `"magned_state"`. A função `persist()` serializa o objeto `S` inteiro via `JSON.stringify()` e grava no browser.
- **Na Nuvem:** Usamos o `Firebase Realtime Database`. A função `syncCloud()` envia o estado para `https://magned-xxxxx.firebaseio.com/users/{userId}.json` via `fetch(url, { method: 'PUT', body: JSON.stringify(data) })`.
- O descritor facial (vetor biométrico) **nunca sai do dispositivo**. Ele só é salvo no `localStorage` e no `Firebase` pessoal do aluno.

 Demonstração: Cadastrar rosto e fazer login ao vivo pela câmera

GA

Gabriel

Motor de IA Híbrido · Fallback Silencioso · Firebase

 Ação: Chat aberto → [Mostra código no GitHub](#)

"O coração do MAGNED é o **Motor Híbrido de IA**. Temos dois caminhos: a **API do Gemini 2.0 Flash** na nuvem e um **Motor Local** em JavaScript puro."

💡 Como funciona o Motor de IA (Gemini + Fallback Local)

- **Caminho 1 — Nuvem (Gemini):** Quando o aluno envia uma mensagem no chat, o sistema chama `fetch('https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent?key=...')`. Enviamos o contexto do Lyceum/AVA + o histórico de conversa + o prompt do tutor ativo.
- **Caminho 2 — Fallback Local:** Se a chamada falhar (timeout, sem internet, cota excedida), o bloco `try/catch` aciona silenciosamente a classe `MagnedAI` do arquivo `magned_ai.js`. O usuário **nunca vê erro** — a resposta simplesmente vem do motor local.
- **Motor Local (magned_ai.js):** É uma classe JavaScript de ~600 linhas com um pipeline: `normalize()` → `classify()` → `dispatch()` → `handler()`. O classificador compara a mensagem normalizada contra ~150 palavras-chave mapeadas para 22 intenções (ex: "nota" → `lyceum_grades`, "quiz" → `quiz`).

🔥 Como funciona o Firebase (Banco em Nuvem)

- Usamos o `Firebase Realtime Database` (NoSQL baseado em JSON).
- A função `syncCloud()` serializa o estado do aluno (nome, CPF, XP, notas, eventos, sessões Pomodoro) e faz um `PUT` no endpoint do Firebase.
- A função `loadFromCloud()` faz um `GET` e mescla os dados da nuvem com os locais usando estratégia **"last-write-wins"** (o dado mais recente prevalece).
- O banco está hospedado no **console.firebase.google.com** — os dados ficam num JSON acessível por URL REST. Não precisamos de SDK pesado.

 Demonstração: Desligar o Wi-Fi e mostrar que o chat continua respondendo



Eduardo

Integração Lyceum · Cálculos Reais · Contexto da IA

 Ação: Aba Clássicas → Lyceum → Pergunta no chat

"A IA só é inteligente se ela tiver **dados reais**. Nós construímos uma ponte entre o Lyceum e o motor de IA."

Como funciona a Integração com o Lyceum

- Os dados do Lyceum ficam no array `S.lyceumData[]`. Cada elemento contém: `disc` (disciplina), `prof` (professor), `n1`, `n2` (notas), `faltas`, `totalAulas` e `status`.
- A função `buildLyceumContext()` percorre esse array e calcula matematicamente: $Média = (N1 + N2) / 2$, $\% \text{ de Falta} = (faltas / totalAulas) \times 100$.
- Se a porcentagem de faltas for $\geq 25\%$, o sistema injeta a tag  **RISCO DE REPROVAÇÃO POR FALTA** no contexto. Se uma nota for < 5.0 , injeta  **NOTA ABAIXO DA MÉDIA**.
- Esse contexto inteiro (nome do aluno, CPF, RA, boletim completo) é **injetado silenciosamente no prompt** antes de cada pergunta que o aluno faz no chat. A IA lê tudo e responde com base nos dados reais.
- A função `renderLyceumGrid()` renderiza os cards visuais na interface com barras de progresso coloridas.

Como funciona a Integração com o AVA

- Os eventos do calendário ficam em `S.events[]`. Cada evento tem: `title`, `date`, `time`, `cat` (categoria: prova, trabalho, etc.).
- A função `buildAVAContext()` filtra os próximos 10 eventos, calcula quantos dias faltam para cada um, e marca como  **URGENTE** (≤ 2 dias) ou  **Esta semana** (≤ 7 dias).
- O aluno pode importar um arquivo `.ics` do AVA real. O parser lê o formato iCalendar e popula `S.events[]`.

 Demonstração: Perguntar "como estão minhas notas?" e ver a IA analisar o boletim real

NI

Nicolas

Sistema de Tutores · Gamificação · Persistência de Estado

 Ação: Troca de tutores → Mostra XP subindo

"A IA do MAGNED não tem uma personalidade só. Nós criamos **6 tutores** com estilos diferentes."

Como funcionam os Tutores com Persona

- Cada tutor é definido por um `name` e um `prompt` de personalidade. A função `setTutor(name, prompt, cardId)` troca a persona ativa.
- O prompt de cada tutor é concatenado com os dados reais do aluno (CPF, RA, nome) antes de cada chamada à IA. Assim, o "Prof. Holehon" responde de um jeito formal e o "MAGNED Padrão" responde de forma mais descontraída — mas ambos conhecem as notas reais do aluno.
- A troca de tutor limpa o histórico de conversa (`S.history = []`) para evitar confusão de contexto.

Como funciona a Gamificação (XP e Níveis)

- A função `gainXP(amount, reason)` incrementa `S.xp` e verifica se o aluno subiu de nível (`S.xp >= S.xpPerLevel`).
- Cada interação gera XP: selecionar matéria (+5), enviar mensagem (+3), completar quiz (+10), sessão Pomodoro (+15).
- O progresso visual é uma barra de XP no perfil da sidebar, renderizada via `Math.min((S.xp / S.xpPerLevel) * 100, 100)` .
- O XP e o nível são persistidos no `localStorage` e sincronizados na nuvem via Firebase.

Como funciona a Persistência de Estado

- O estado inteiro da aplicação vive num único objeto JavaScript chamado `S` (State).
- Sempre que algo muda (nova mensagem, novo evento, XP ganho), chamamos `persist()` que faz `localStorage.setItem('magned_state', JSON.stringify(S))` .
- No carregamento da página, `loadState()` lê o `localStorage` e hidrata o objeto `S` de volta. É por isso que seus dados nunca somem ao fechar o navegador.

 Demonstração: Trocar de tutor e mostrar a barra de XP subindo

DA

Davi

Quizzes · Flashcards · Mapas Mentais — Estudo Ativo

 Ação: Pedir "gerar quiz de Python" no chat

"Nosso diferencial é que a IA não cospe só texto. Ela gera **componentes visuais interativos**."

Como funcionam os Quizzes

- O arquivo `magned_ai.js` contém um banco de 30+ questões distribuídas em 6 áreas (Python, Banco de Dados, IA, Redes, Engenharia, Lógica).
- Quando o aluno digita "quiz de python", o motor classifica a intenção como `quiz`, seleciona a matéria e sorteia 5 questões aleatórias via `Math.random()`.
- A resposta é retornada como **HTML puro** com botões interativos. Cada botão tem um `onclick` que compara a resposta selecionada com o gabarito e colore verde (correto) ou vermelho (errado).
- O quiz é injetado diretamente no chat via `innerHTML`, criando widgets clicáveis dentro da conversa.

Como funcionam os Flashcards

- A IA gera flashcards como pares de `{frente, verso}` em HTML.
- A animação de flip usa **CSS 3D Transforms**: `transform: rotateY(180deg)` com `backface-visibility: hidden`.
- Os flashcards são armazenados em `S.flashcards[]` e navegados com `S.fcIdx` (índice atual). Setas para frente/trás incrementam/decrementam o índice.

Como funcionam os Mapas Mentais

- O motor local contém templates de mapas mentais em formato **ASCII/diagrama** para cada matéria.
- Ao receber a intenção `mapa_mental`, o handler retorna o diagrama formatado com caracteres de caixa (`┌ ─┐`) dentro de um bloco `<pre>`.
- Isso permite que o aluno **visualize hierarquias** de conceitos de forma rápida e organizada.

 Demonstração 1: "Gerar quiz de Python" → Responder questões clicando Demonstração 2: "Mapa mental de Banco de Dados" → Diagrama visual

MA

Marcel

OCR Vision · PWA · Pomodoro · GitHub

 Ação: Ferramentas → OCR → Mostrar GitHub

"Para fechar as features, vou explicar 3 coisas que ninguém espera num projeto acadêmico: **OCR**, **PWA** e **Pomodoro**."

Como funciona o OCR (Visão Neural)

- Usamos a biblioteca `Tesseract.js v5`, que é uma compilação WebAssembly do Tesseract (o motor de OCR open-source do Google).
- O aluno anexa uma foto (do quadro, caderno, ou livro) via `<input type="file">`.
- O `Tesseract.recognize(image, 'por')` processa a imagem pixel a pixel, identifica caracteres e retorna o texto extraído.
- O texto extraído é automaticamente enviado como prompt para a IA, que resolve o exercício ou explica o conteúdo da foto.
- Uma barra de progresso visual mostra o andamento do OCR em tempo real.

Como funciona o PWA (Progressive Web App)

- `manifest.json`: Define o nome do app, ícone, cores do tema e `display: standalone` (abre sem barra de navegação).
- `sw.js` (Service Worker): No evento `install`, faz cache de todos os arquivos críticos (`index.html`, `app.html`, `magned_ai.js`). No evento `fetch`, intercepta requisições e serve do cache quando offline.
- Resultado: o aluno pode **instalar o MAGNED** no celular ou desktop como um app nativo. Ele funciona mesmo sem internet porque o Service Worker serve os arquivos do cache local.

Como funciona o Pomodoro

- Timer de 25 minutos (foco) + 5 minutos (pausa), controlado por `setInterval()` que decrementa `S.pomSec` a cada segundo.
- Um anel visual animado (SVG com `stroke-dasharray`) mostra o progresso circular.
- Ao finalizar, uma `Notification API` nativa do navegador avisa o aluno e ganha XP automaticamente.

Apresentação do GitHub

- Repositório: github.com/GRUPO01-bit/MAGNED
- Mostrar os commits recentes, o histórico de evolução (Ciclo 1 → 2 → 3), e os arquivos organizados.
- O professor pode clonar e rodar com um simples duplo-clique no `index.html`.

 Demonstração: Anexar foto → OCR extrai texto → IA resolve

"Muito obrigado! Estamos abertos a perguntas sobre qualquer parte da arquitetura." 